

Agent开发系列

从用户到创造者 · 30篇完整收录

用代码理解AI · 电子书版

公众号：大前端工程师
公众号：大前端工程师
公众号：大前端工程师

最小可行Agent

上一期我们说了Agent的核心循环：感知、思考、行动、反思。(见[OpenClaw原理篇11](#))

光说不练假把式。这一期，我们写一个能跑的系统。代码不到100行，但该有的循环都有。

💡 **先跑起来：**你看10篇原理文章，不如亲手写一个能跑的系统有感觉。

1 最简版本：30行代码

先去掉所有花哨的东西，就保留最核心的循环：

```
import openai

class SimpleAgent:
    def __init__(self):
        self.messages = []

    def run(self, user_input):
        # 感知：把用户输入加到对话历史
        self.messages.append({"role": "user", "content": user_input})

        # 思考+行动：调用大模型
        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=self.messages
        )

        # 反思：从响应中提取助手回复
        assistant_msg = response.choices[0].message
        self.messages.append(assistant_msg)

        return assistant_msg["content"]
```

就这么点代码。一个类，三个方法，加起来不到30行。用法：

```
agent = SimpleAgent()
print(agent.run("你好"))
print(agent.run("帮我写一首诗"))
```

这个版本能跑，但它只是个对话机器人。[Agent和普通聊天机器人的区别](#)，在于那个“循环”。[\(核心循环详解\)](#)

2 进阶版本：加工具调用

真正的Agent会自己决定要不要调用工具。[\(参考OpenClaw进阶篇09\)](#)

```
class SimpleAgentWithTools:
    def __init__(self):
        self.messages = []
        self.tools = {}

    def register_tool(self, name, func):
        self.tools[name] = func

    def run(self, user_input, max_turns=5):
        self.messages.append({"role": "user", "content": user_input})

        for _ in range(max_turns):
            response = openai.ChatCompletion.create(
                model="gpt-4",
                messages=self.messages,
                tools=self.tools
            )

            message = response.choices[0].message

            if message.get("tool_calls"):
                for call in message["tool_calls"]:
```

```
        tool_name = call["function"]["name"]
        args = json.loads(call["function"]["arguments"])
        result = self.tools[tool_name](**args)
        self.messages.append({
            "role": "tool",
            "tool_call_id": call["id"],
            "content": str(result)
        })
    else:
        self.messages.append(message)
        return message["content"]

    return "已达到最大轮次"
```

新增三个关键点：

- register_tool — 注册工具，让Agent知道有哪些武器可用
- max_turns — 限制最大轮次，防止无限循环
- tool_calls — 检测模型是否想调用工具 ([参考AI Coding第28篇](#))

举个例子。用户问"帮我算 $(12 + 8) * 3$ "：

1. Agent想了想，发现要调用calculator
2. 调用工具，算出60
3. 把结果返回给用户

💡 **这就是Agent的核心能力：**不只是聊天，而是能真的帮你做事。

3 还缺什么

这个最小版本能跑，但生产环境还差得远：

- ✗ 记忆系统 – 每次对话从零开始，Agent记不住之前说过什么（见OpenClaw原理篇12）
- ✗ 反思机制 – 没有判断结果对不对的能力
- ✗ 错误处理 – 工具调用失败了怎么办
- ✗ 流式输出 – 等全部生成完再返回，用户体验差

这个系列的后续文章，会一个一个把这些补上。先记住这个框架，后面的东西都好说。

感知：用户输入到结构化意图

感知是什么？你跟Agent说"帮我订明天去上海的机票"，Agent得先理解：

- 用户想要什么？→ 订机票
- 出发地？→ 没说，可能是当前位置
- 目的地？→ 上海

这个理解的过程，就是感知。**把用户的自然语言，变成机器能理解的结构化数据。**

💡 **感知是第一步：**没有这一步，Agent就是个复读机——你说啥它重复啥，不知道你要什么。

1 意图识别

用户说了一句话，Agent首先要判断：用户想要做什么？这个过程叫意图识别（Intent Detection）。([参考AI Coding第28篇](#))



PYTHON

```
INTENTS = ["订机票", "查天气", "写代码", "搜索信息", "其他"]

def detect_intent(user_input):
    prompt = f"用户说: \"{user_input}\" \n这个用户的意图是什么? "

    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}]
    )

    return response.choices[0].message["content"]
```

⚠ 但这个方法太慢了——每次都要调一次大模型。实际项目中，可以用规则匹配或者小模型来做意图识别，把大模型留到真正需要的地方。

2 实体提取

判断了意图之后，还要提取关键信息。比如用户说“帮我订后天北京到上海的机票”：

- 时间 → 后天
- 出发地 → 北京
- 目的地 → 上海

这个过程叫实体提取（Entity Extraction）。代码：

```
def extract_entities(user_input):
    prompt = f"""从以下用户输入中提取关键信息：

    用户输入: \"{user_input}\"

    提取以下信息（如果没有则填“未知”）：
    - 时间
    - 出发地
    - 目的地
    - 人数
```

请用JSON格式返回"""

```
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)

import json
return json.loads(response.choices[0].message["content"])
```

3 整合：感知的完整实现

```
def perceive(self, user_input):
    # 1. 识别意图
    intent = self.detect_intent(user_input)

    # 2. 提取实体
    entities = self.extract_entities(user_input)

    # 3. 组合成结构化上下文
    return {
        "intent": intent,
        "entities": entities,
        "raw_input": user_input
    }
```

💡 **感知做好了：**后面的思考和行动就轻松了。Agent知道你要什么，才能决定怎么做。

4 下一步

感知知道了你要什么，思考来决定怎么做。下一步我们来说说思考。

有问题欢迎留言。

相关系列阅读

- OpenClaw原理篇11：核心循环 {原文件:11-核心循环-公众号版.md}
- OpenClaw原理篇12：记忆系统 {原文件:12-记忆系统-公众号版.md}
- OpenClaw进阶篇09：工具与API集成 {原文件:09-工具与API集成-公众号版.md}
- AI Coding第28篇：Agent核心实现 {原文件:AI-28-Agent核心实现-公众号版.md}
- AI Coding第29篇：工具系统 {原文件:AI-29-工具系统-Tool-System-公众号版.md}

下期预告：Agent开发系列 05-06 — 思考：ReAct循环 + 行动：工具调用

Agent开发系列 · 用代码理解AI

思考：Agent是怎么"想"的

上一期我们说了感知。用户说了一句话，Agent知道用户想要什么了。

接下来是思考。感知知道了你要什么，思考来决定怎么做。(见[OpenClaw原理篇11](#))

思考听起来很玄乎。但在大模型时代，思考其实就是一次大模型的调用。模型根据你的问题和上下文，决定下一步该做什么。

💡 **核心观点：**所谓的"思考"，本质上是让大模型根据当前情况，决定是调用工具、收集更多信息，还是直接回答。

1 ReAct模式

最常用的思考模式叫ReAct——Reasoning + Acting。就是一边推理，一边行动。

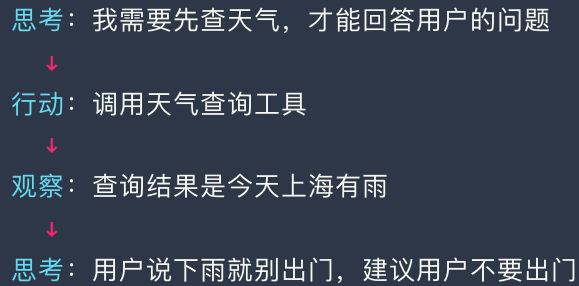
举个例子。用户问："帮我查一下上海今天的天气，如果下雨就别出门了。"

Agent的思考过程是这样的：

1. 思考：我需要先查天气，才能回答用户的问题
2. 行动：调用天气查询工具
3. 观察：查询结果是今天上海有雨
4. 思考：用户说下雨就别出门，我应该建议用户今天不要出门



Figure: ReAct Loop - Thought -> Action -> Observation -> Thought



思考：我需要先查天气，才能回答用户的问题

↓

行动：调用天气查询工具

↓

观察：查询结果是今天上海有雨

↓

思考：用户说下雨就别出门，建议用户不要出门

2 代码实现

([参考AI Coding第28篇](#))



```
def think(self, context):
    prompt = f"""当前任务: {context.task}
    已有信息: {context.observations}

    请思考: 下一步应该做什么?
    选项:
    1. 继续收集信息 (调用工具)
    2. 直接回答用户
    3. 任务已完成

    请用JSON格式返回你的思考和决定。

    {{
        "thought": "你的思考过程",
        "action": "下一步行动",
        "args": "行动参数"
    }}"""

    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}]
    )
```

```
import json
return json.loads(response.choices[0].message["content"])
```

3 Chain of Thought

还有一个概念叫Chain of Thought，思维链。就是让模型把思考过程说出来，而不只是给结论。

💡 **实践经验：**让模型"解释一下你是怎么想的"，往往能得到更准确的答案。

知道了Agent是怎么"想"的，接下来我们来说说它是怎么"做"的——行动。

行动：工具是怎么被调用的

思考完了，下一步就是行动。行动就是调用工具。[\(参考OpenClaw进阶篇09\)](#)

💡 **核心观点：**Agent能做什么事，就看它有多少工具。工具越多，能力越强。[\(参考AI Coding第29篇\)](#)

1 Function Calling

现在主流的Agent都支持Function Calling。就是让模型在生成内容的时候，顺便告诉我"我需要调用某个函数"。



PYTHON

```
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "帮我查一下上海天气"}],
    tools=[
        {
            "type": "function",
            "function": {
                "name": "get_weather",
                "description": "查询天气",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "city": {
                            "type": "string",
                            "description": "城市名称"
                        }
                    }
                },
                "required": ["city"]
            }
        }
    ]
)

# 检查是否有工具调用
if response.choices[0].message.get("tool_calls"):
    tool_call = response.choices[0].message["tool_calls"][0]
    function_name = tool_call["function"]["name"]
    arguments = json.loads(tool_call["function"]["arguments"])
    print(f"需要调用函数: {function_name}")
    print(f"参数: {arguments}")
```

2 工具执行

拿到函数名和参数之后，下一步就是执行：

```
def execute_tool(self, tool_name, args):
    tool_func = self.tools.get(tool_name)

    if tool_func is None:
```

```
        return f"找不到工具: {tool_name}"

    try:
        result = tool_func(**args)
        return result
    except Exception as e:
        return f"工具执行出错: {str(e)}"
```

3 结果返回

工具执行完了，要把结果传回给模型，让模型决定下一步怎么做：

PYTHON

```
# 执行工具
tool_result = self.execute_tool(function_name, arguments)

# 把结果加到消息历史
self.messages.append({
    "role": "tool",
    "tool_call_id": tool_call["id"],
    "content": str(tool_result)
})

# 再次调用模型，让它处理工具返回的结果
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=self.messages
)
```

 **完整流程：**模型决定调用 → 传递参数 → 执行工具 → 返回结果 → 模型继续处理。

4 常见的内置工具

工具	作用	例子
搜索	搜索互联网	查天气、查股票

工具	作用	例子
计算	执行计算	数学运算
文件	读写文件	保存结果、读取配置
API	调用外部接口	发邮件、发消息
代码	执行代码	运行Python脚本

 **注意事项：** 工具调用不是越多越好。频繁的上下文切换会影响Agent的工作效率。设计工具时要考虑实用性，避免过度设计。

下一步我们来说说反思。工具调用完了，怎么知道结果对不对？这就是反思要解决的问题。

有问题欢迎留言。

 相关系列阅读

- OpenClaw原理篇11：核心循环 {原文件:11-核心循环-公众号版.md}
- OpenClaw进阶篇09：工具与API集成 {原文件:09-工具与API集成-公众号版.md}
- AI Coding第28篇：Agent核心实现 {原文件:AI-28-Agent核心实现-公众号版.md}
- AI Coding第29篇：工具系统 {原文件:AI-29-工具系统-Tool-System-公众号版.md}

下期预告：Agent开发系列 07-08 — 反思 + 记忆系统

反思：Agent是怎么"反省"的

上一期我们说了行动。Agent调用了工具，拿到了结果。

然后呢？然后就结束了吗？

不是。Agent还需要反思：这次做得怎么样？结果对不对？要不要重来一次？(见[OpenClaw 原理篇11](#))。

💡 **核心观点：**反思是Agent自我纠错的关键。没有反思，Agent就会"一条道走到黑"，不管结果对不对都继续执行。

1 为什么要反思

举个例子。用户让Agent帮他写一封邮件。Agent调用工具写完了，把结果返回给用户。

但用户可能不满意——语气太正式了，或者遗漏了什么信息。

如果没有反思环节，Agent就只会机械地执行任务，不会自我改进。加上反思，Agent才能知道自己的不足，然后调整策略、重新尝试。

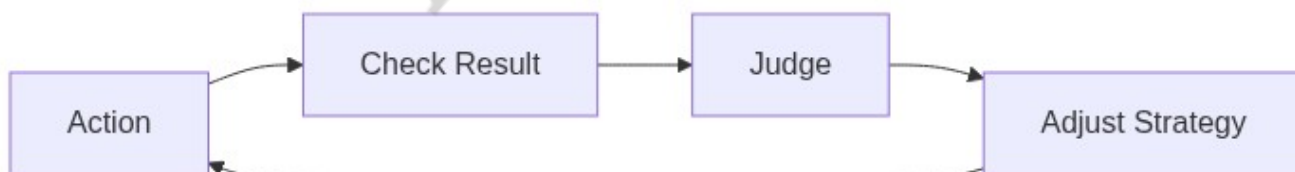


Figure: Agent Reflection Flow - Action -> Check -> Judge -> Adjust

2 代码实现

反思的核心是判断结果是否正确。简单来说，就是问自己几个问题：

- 工具调用成功了吗？
- 返回的结果合理吗？
- 用户的问题解决了吗？
- 要不要重试？

代码大概是这样：

```
PYTHON

def reflect(self, context, result):
    if result.is_error:
        return {
            "success": False,
            "reason": "工具调用出错",
            "retry": True,
            "strategy": "换工具或换参数"
        }

    if not self.validate_result(result):
        return {
            "success": False,
            "reason": "结果验证失败",
            "retry": True,
            "strategy": "调整参数重试"
        }

    if not self.is_task_done(context, result):
        return {
            "success": False,
            "reason": "任务未完成",
            "retry": True,
            "strategy": "继续执行下一步"
        }

    return {
        "success": True,
        "reason": "任务完成",
        "retry": False,
```

```
"strategy": None
}
```

💡 **实践要点：**反思就是让Agent"回头看"：这次做得怎么样？要不要调整？要不要重试？

3 反思的粒度

反思可以做得很简单，也可以做得复杂。

级别	做法	适用场景
简单版	检查工具调用有没有报错	快速原型、简单任务
中等版	检查返回结果是否合理	一般业务场景
复杂版	让大模型判断结果质量	对准确性要求高的任务

⚠️ **建议：**实际项目中，可以先做简单版，再慢慢加复杂逻辑。过早追求复杂的反思机制，往往是过度设计。

说完反思，接下来我们来说说记忆系统——Agent是怎么记住之前说过的话的。

记忆系统：Agent是怎么"记住"东西的

之前我们写的最小可行Agent，有个问题：每次对话都是从零开始。[\(见OpenClaw原理篇12\)](#)

你跟它说"帮我写一封邮件"，它写完了。你再说"把语气改得更正式一点"，它不知道你说的是哪封邮件，因为它记不住。

记忆系统就是来解决这个问题的。Agent需要记住之前说过的话、做过的事，这样才能连续对话。

💡 **核心观点：**没有记忆的Agent，每次对话都是"陌生人"。记忆让Agent能认识"回头客"。

1 四层记忆架构

一般来说，Agent的记忆分四层：[\(参考AI Coding第28篇\)](#)

第一层：对话历史（短期记忆）

记录当前对话的内容。对话结束就消失。每次API调用都会带上。

第二层：实体记忆（中期记忆）

记录用户的基本信息，比如姓名、偏好、常驻城市。跨对话保持。

第三层：知识记忆（长期记忆）

Agent从训练中学会的知识，或者从外部检索来的信息。

第四层：技能记忆

Agent学会的技能，比如怎么写代码、怎么分析数据。

2 代码实现

最简单的记忆系统，就是把对话历史存起来，每次调用的时候带上：



PYTHON

```
class AgentWithMemory:
    def __init__(self):
        self.messages = []
        self.entities = {}

    def add_message(self, role, content):
        self.messages.append({"role": role, "content": content})

    def add_entity(self, key, value):
        self.entities[key] = value

    def build_context(self):
        context = []

        if self.entities:
            entity_prompt = "你知道以下关于用户的信息: "
            for key, value in self.entities.items():
                entity_prompt += f"\n- {key}: {value}"
            context.append({"role": "system", "content": entity_prompt})

        context.extend(self.messages)
        return context
```

用起来是这样的：

```
PYTHON

agent = AgentWithMemory()

agent.add_entity("name", "张三")

agent.add_message("user", "帮我订明天去上海的机票")
agent.add_message("assistant", "好的，张三，我来帮你查...")

agent.add_message("user", "改成后天的")
# Agent现在知道用户是张三，知道之前订的是上海机票
```

💡 **实践要点：**记忆系统让Agent能“记住”用户说过的话、做过的操作。这样它才能真正帮你完成任务，而不是每次都从零开始。

3

记忆的管理

记忆多了会有问题。上下文窗口是有限的，存太多东西API会爆。

策略	做法	适用场景
定期压缩	把多轮对话压缩成摘要	长对话场景
重要性筛选	只保留重要的信息	信息繁多的场景
向量检索	把记忆向量化，需要时检索	大规模记忆存储
分层存储	短期记忆放内存，长期记忆放数据库	需要持久化的场景

 **注意事项：**不是记忆越多越好。有效的记忆管理比盲目存储更重要。

到这里，Agent的核心循环我们就全讲完了——感知、思考、行动、反思，再加上记忆系统。

有问题欢迎留言。

相关系列阅读

- OpenClaw原理篇11：核心循环 {原文件:11-核心循环-公众号版.md}
- OpenClaw原理篇12：记忆系统 {原文件:12-记忆系统-公众号版.md}
- AI Coding第28篇：Agent核心实现 {原文件:AI-28-Agent核心实现-公众号版.md}

下期预告：Agent开发系列 09-10 — 规划系统 + 多Agent协作

核心循环篇总结：Agent是怎么"思考"的

感知、思考、行动、反思——这四个环节，构成了Agent的核心循环。[\(参考OpenClaw原理篇11\)](#)

这一篇，我们把前几期的内容串起来，画一张完整的全景图。

💡 **核心观点：** Agent不是什么魔法，而是一个"感知→思考→行动→反思"的循环。理解了 this 循环，就理解了Agent的本质。

1 核心循环全景图

先来看一张完整的流程图：

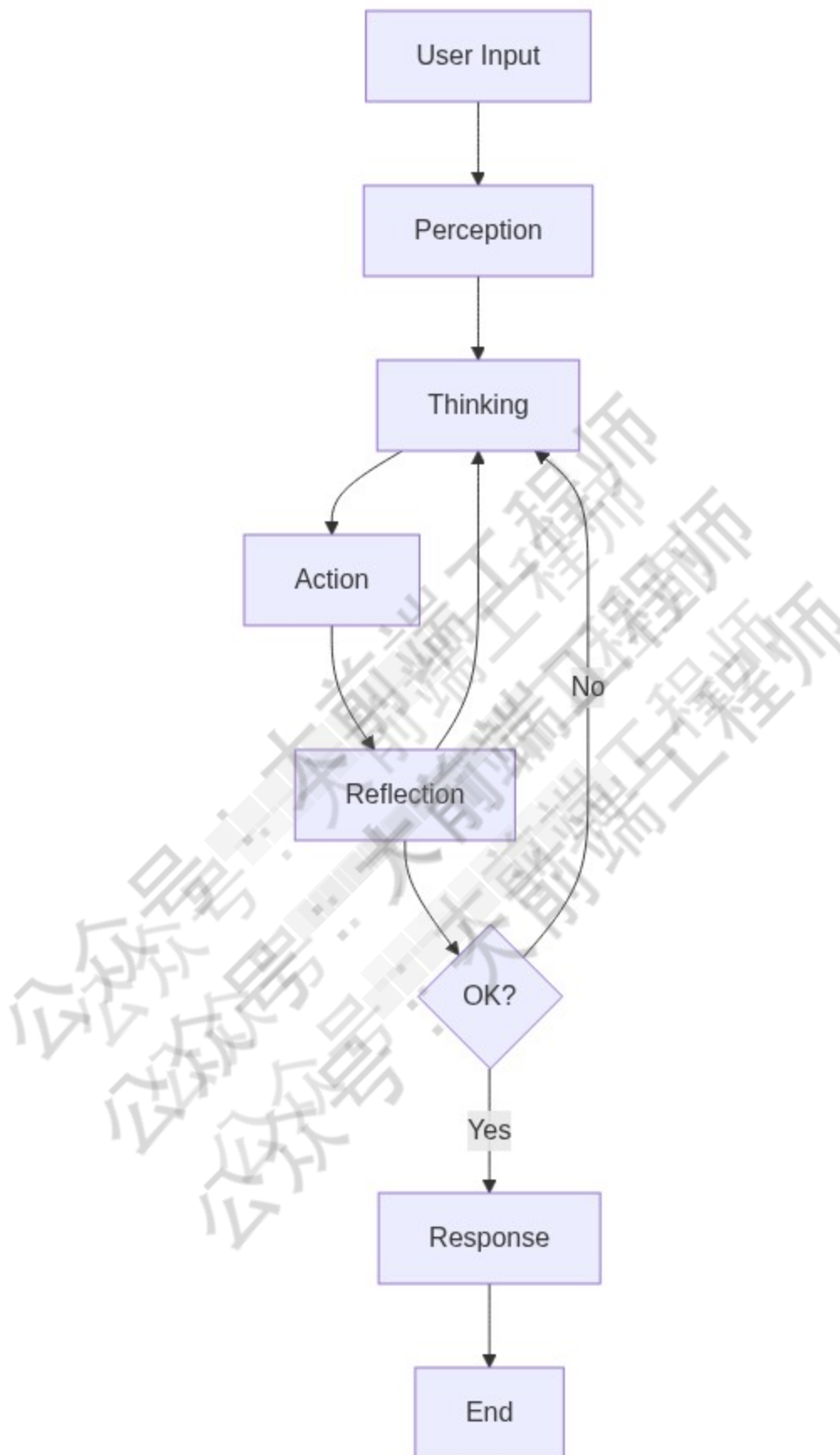
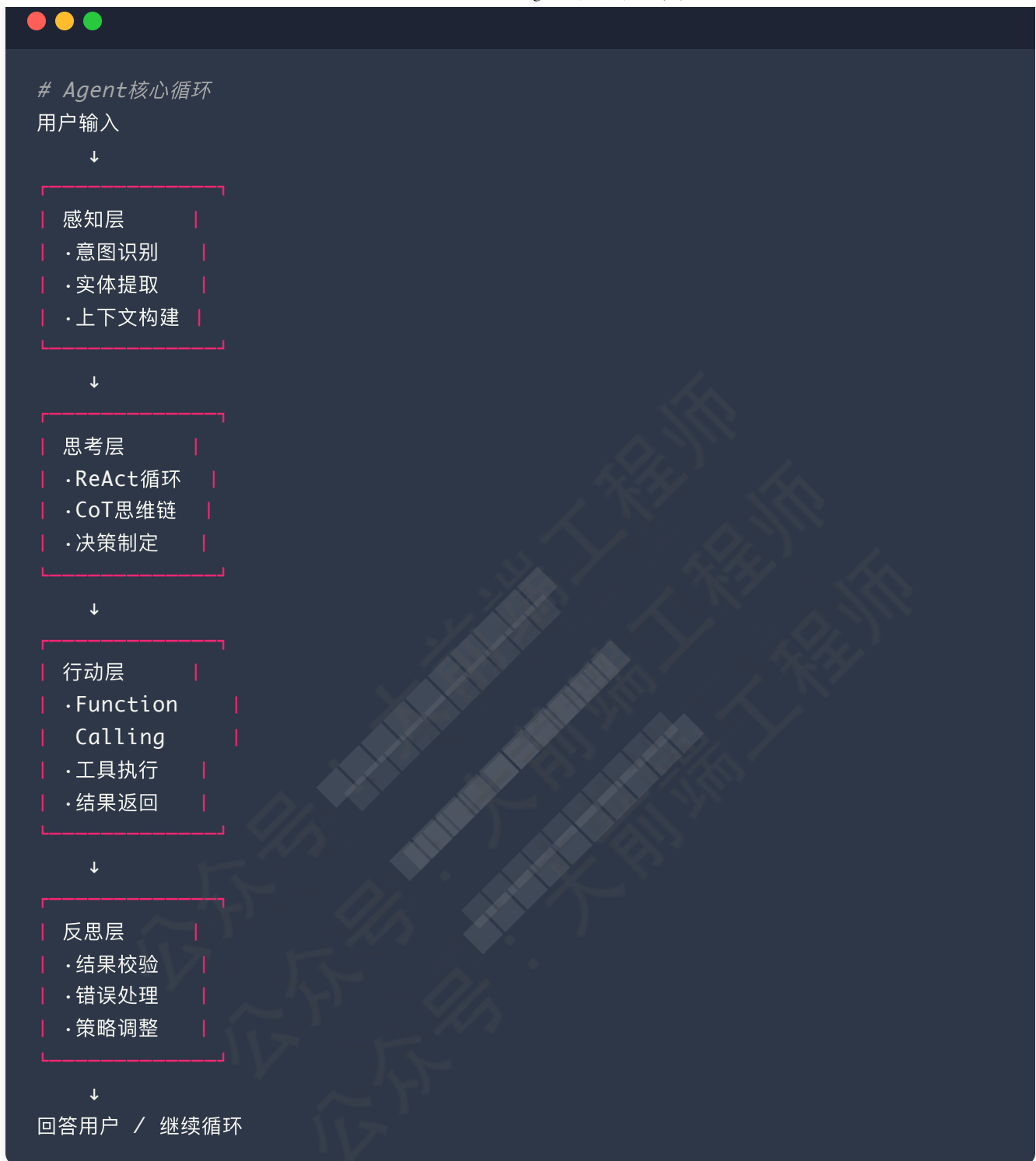


Figure: Agent Core Loop - Perception -> Thinking -> Action -> Reflection



这是一个循环，不是一次性的流程。Agent会不断在思考层和行动层之间循环，直到任务完成或者达到最大循环次数。

2 四层职责回顾

层次	输入	输出	核心问题
感知	用户原始输入	结构化意图+实体	用户真正想要什么？
思考	意图+上下文	行动决策	我应该怎么做？
行动	决策+工具列表	执行结果	工具返回了什么？
反思	执行结果	校验+策略调整	结果对吗？需要重试吗？

3

ReAct循环：思考与行动的交替

ReAct是现在最主流的Agent思考模式。[\(参考AI Coding第28篇\)](#)

它的核心是：Thought → Action → Observation → Thought → Action...

PYTHON

```
async def run(self, user_input: str, max_turns: int = 10):
    messages = [{"role": "user", "content": user_input}]

    for turn in range(max_turns):
        # 1. 思考：让模型决定下一步
        response = await self.llm.chat(messages, tools=self.tools)

        # 2. 如果没有工具调用，直接返回
        if not response.tool_calls:
            messages.append(response)
            return response.content

        # 3. 执行工具
        for tool_call in response.tool_calls:
            result = await self.llm.execute_tool(tool_call)
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": str(result)
            })

        # 4. 反思：检查是否需要继续
```

```
if self.is_complete(messages):  
    break  
  
return "已达到最大循环次数"
```

这个循环会一直持续，直到：

- 模型认为任务完成，不再调用工具
- 达到最大循环次数（防止无限循环）
- 出现无法恢复的错误

4 工具调用是桥梁

思考和行动之间，工具调用是桥梁。[\(参考OpenClaw进阶篇09\)](#)

模型生成tool_calls，我们负责执行，然后把结果传回去。这个设计非常精妙：

- 模型不需要知道工具怎么执行——只管"调用"就行
- 我们不需要写复杂的决策逻辑——让模型自己决定
- 扩展性极强——加一个新工具，模型就能用

```
# 工具注册  
self.tools = [  
    {  
        "type": "function",  
        "function": {  
            "name": "get_weather",  
            "description": "查询城市天气",  
            "parameters": { "type": "object", "properties": {...} }  
        }  
    },  
    # 更多工具...  
]
```

5 反思让Agent更可靠

没有反思的Agent，就像一个不会复盘的人——犯了错不知道改，同样的错误会一直犯。(参考[AI Coding第29篇](#))

反思层要做的事情：

1. **结果校验**——工具返回的结果是否符合预期？
2. **错误处理**——出错了是重试还是换策略？
3. **策略调整**——这个方法不行，要不要试试别的？

```
PYTHON

def reflect(self, tool_result, context):
    # 检查工具执行是否出错
    if is_error(tool_result):
        if is_retryable(tool_result):
            return "retry" # 可重试的错误，重试一次
        else:
            return "fail" # 不可重试的错误，直接失败

    # 检查结果是否符合预期
    if not self.is_satisfactory(tool_result, context):
        return "replan" # 结果不满意，需要重新规划

    return "continue" # 一切正常，继续执行
```

6 核心循环的意义

理解了核心循环，你能做到很多事情：

- **调试Agent**——当Agent行为不符合预期，你能定位是哪一层出了问题
- **优化Agent**——知道哪层薄弱，就重点优化哪层
- **设计新Agent**——不管什么场景，都可以用这个框架来思考

💡 **总结：** Agent的核心循环是"感知→思考→行动→反思"的无限循环。ReAct模式让模型自己决定下一步做什么。工具调用是思考和行动的桥梁。反思让Agent能自我纠错。这个循环理解透了，Agent开发就学会了一半。

有问题欢迎留言。

📎 相关系列阅读

- [OpenClaw原理篇11：核心循环](#)
- [OpenClaw进阶篇09：工具与API集成](#)
- [AI Coding第28篇：Agent核心实现](#)
- [AI Coding第29篇：工具系统](#)

下期预告：Agent开发系列 09-10 — 记忆系统篇

Agent开发系列 · 用代码理解AI

四层记忆架构

上一期我们说了反思。Agent能判断结果对不对，能调整策略。

但还有一个问题没解决：Agent是怎么记住之前说过的话的？(见[OpenClaw原理篇12](#))

这就是记忆系统要解决的问题。

💡 **核心观点：**没有记忆的Agent，每次对话都是"陌生人"。记忆让Agent能认识"回头客"。

1 为什么需要记忆

之前我们写的最小可行Agent，有个问题：每次对话都是从零开始。

你跟它说"帮我写一封邮件"，它写完了。你再说"把语气改得更正式一点"，它不知道你说的是哪封邮件，因为它记不住。

记忆系统就是来解决这个问题的。Agent需要记住之前说过的话、做过的事，这样才能连续对话。

2 四层记忆架构

一般来说，Agent的记忆分四层：(参考[AI Coding第28篇](#))

第一层：对话历史（短期记忆）

记录当前对话的内容。对话结束就消失。每次API调用都会带上。

第二层：实体记忆（中期记忆）

记录用户的基本信息，比如姓名、偏好、常驻城市。跨对话保持。

第三层：知识记忆（长期记忆）

Agent从训练中学会的知识，或者从外部检索来的信息。

第四层：技能记忆

Agent学会的技能，比如怎么写代码、怎么分析数据。

3 代码实现

最简单的记忆系统，就是把对话历史存起来，每次调用的时候带上：

```
PYTHON

class AgentWithMemory:
    def __init__(self):
        self.messages = []
        self.entities = {}

    def add_message(self, role, content):
        self.messages.append({"role": role, "content": content})

    def add_entity(self, key, value):
        self.entities[key] = value

    def build_context(self):
        context = []

        if self.entities:
            entity_prompt = "你知道以下关于用户的信息："
            for key, value in self.entities.items():
                entity_prompt += f"\n- {key}: {value}"
            context.append({"role": "system", "content": entity_prompt})

        context.extend(self.messages)
        return context
```

用起来是这样的：

```
agent = AgentWithMemory()

agent.add_entity("name", "张三")

agent.add_message("user", "帮我订明天去上海的机票")
agent.add_message("assistant", "好的，张三，我来帮你查...")

agent.add_message("user", "改成后天的")
# Agent现在知道用户是张三，知道之前订的是上海机票
```

 **实践要点：**记忆系统让Agent能"记住"用户说过的话，做过的操作。这样它才能真正帮你完成任务，而不是每次都从零开始。

4 记忆的管理

记忆多了会有问题。上下文窗口是有限的，存太多东西API会爆。

策略	做法	适用场景
定期压缩	把多轮对话压缩成摘要	长对话场景
重要性筛选	只保留重要的信息	信息繁多的场景
向量检索	把记忆向量化，需要时检索	大规模记忆存储
分层存储	短期记忆放内存，长期记忆放数据库	需要持久化的场景

 **注意事项：**不是记忆越多越好。有效的记忆管理比盲目存储更重要。

说完四层记忆架构，接下来我们来说说具体怎么管理这些记忆——上下文窗口管理。

上下文窗口管理

记忆系统让Agent能记住东西了。但还有一个问题：上下文窗口是有限的。

GPT-4的上下文窗口有128K tokens听起来很大，但实际用起来很快就会塞满。特别是处理长文档、进行多轮对话的时候。

💡 **核心观点：** 上下文窗口管理，就是在有限的窗口内塞进最多的有效信息。

1 滑动窗口

最简单的方法是滑动窗口。只保留最近N条消息，超出的就丢掉。

```
PYTHON

class SlidingWindowMemory:
    def __init__(self, max_messages=10):
        self.messages = []
        self.max_messages = max_messages

    def add(self, role, content):
        self.messages.append({"role": role, "content": content})

        if len(self.messages) > self.max_messages:
            self.messages = self.messages[-self.max_messages:]

    def get_messages(self):
        return self.messages
```

缺点也很明显：早天的对话会被完全丢掉。如果用户说"接着上次说的继续写"，Agent可能完全不记得上次说了什么。

2 摘要压缩

更好的方法是摘要压缩。当对话太长时，把之前的对话压缩成摘要。

```
PYTHON

class SummarizingMemory:
    def __init__(self, max_tokens=4000):
        self.summary = ""
        self.recent_messages = []
        self.max_tokens = max_tokens

    def add(self, role, content):
        self.recent_messages.append({"role": role, "content": content})

        if self.count_tokens(self.recent_messages) > self.max_tokens:
            self.compress()

    def compress(self):
        prompt = f"""请把以下对话压缩成一段摘要：
{self.recent_messages}
摘要应该包含：1) 主要话题 2) 用户的关键需求 3) 任何重要决定或结论"""

        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=[{"role": "user", "content": prompt}]
        )

        self.summary = response.choices[0].message["content"]
        self.recent_messages = []

    def get_context(self):
        context = []
        if self.summary:
            context.append({"role": "system", "content": f"之前的对话摘要：{self.summary}"})
        context.extend(self.recent_messages)
        return context
```

3 Token优化

除了压缩，还有一些小技巧可以节省token：

技巧	做法	节省比例
缩短系统提示	把重复性的指示压缩	10-20%
精简返回格式	让模型返回更简洁	5-15%
移除冗余	删除对话中的客套话	5-10%
使用更短的模式	简单任务用GPT-3.5	可达50%

💡 **实践建议：**先用滑动窗口，效果不好再上摘要压缩。复杂方法不一定带来更好的效果。

到这里，记忆系统的核心就讲完了。下一篇我们来说说向量数据库——一种更强大的记忆检索方式。

有问题欢迎留言。

📎 相关系列阅读

- OpenClaw原理篇12：记忆系统 {原文件:12-记忆系统-公众号版.md}
- AI Coding第28篇：Agent核心实现 {原文件:AI-28-Agent核心实现-公众号版.md}

下期预告：Agent开发系列 11-12 — 向量数据库与RAG + 实战：让Agent记住你是谁

Agent开发系列 · 用代码理解AI

向量数据库与RAG

上一期我们说了上下文窗口管理。滑动窗口、摘要压缩，这些方法能解决一部分问题。

但对于大规模知识的存储和检索，这些方法还不够。想象一下：如果要让Agent读完一整本书再回答问题，滑动窗口肯定兜不住。

这时候就需要向量数据库和RAG了。

💡 **核心观点：** RAG = Retrieval Augmented Generation，即"检索增强生成"。让Agent先检索相关信息，再基于这些信息生成答案。

1 Embedding是什么

在说向量数据库之前，先说说什么是Embedding。

Embedding就是把你写的文字，变成一串数字（向量）。相似的文字，会变成相似的向量。

```
PYTHON

import openai

response = openai.Embedding.create(
    input="今天天气真好",
    model="text-embedding-ada-002"
)

vector = response['data'][0]['embedding']
print(len(vector))  # 1536 维向量
```

"今天天气真好"和"外面阳光灿烂"，虽然字不一样，但意思相近，所以它们的向量也会比较

接近。

2 向量数据库

向量数据库就是用来存储这些向量的。存储之后，可以根据"相似度"来检索。

常见的向量数据库有：

- ChromaDB（轻量级，适合本地）
- Pinecone（云服务，适合生产）
- Milvus（开源，可私有部署）
- Weaviate（开源，功能丰富）

用ChromaDB举个例子：



```
import chromadb

client = chromadb.Client()
collection = client.create_collection("knowledge_base")

documents = [
    "Python是一门编程语言",
    "JavaScript主要用于Web开发",
    "机器学习是AI的一个分支",
]

collection.add(
    documents=documents,
    ids=["doc1", "doc2", "doc3"]
)

results = collection.query(
    query_texts=["什么是编程"],
    n_results=2
```

)

```
print(results)
```

3 RAG实战

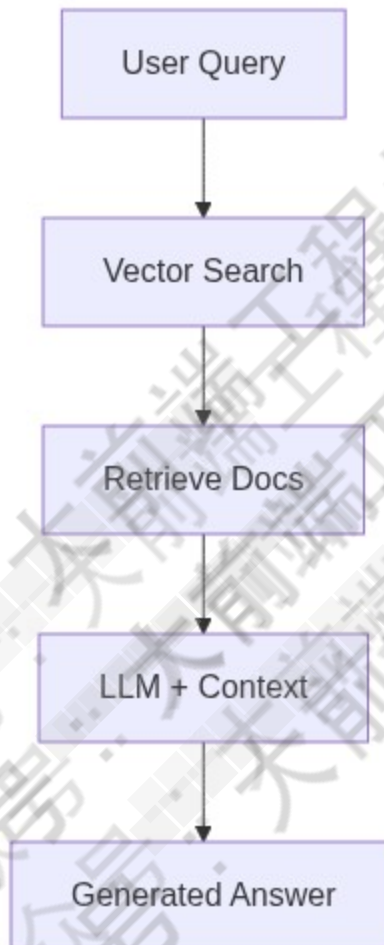


Figure: RAG Flow - Query -> Vector Search -> Retrieve -> LLM -> Answer

RAG的流程是这样的：

1. 把知识文档切成小块，存入向量数据库
2. 用户提问时，先在向量数据库里检索相关的内容
3. 把检索到的内容和用户问题一起发给大模型
4. 大模型基于这些内容生成答案

```
class RAGAgent:
    def __init__(self, vector_store):
        self.vector_store = vector_store

    def retrieve(self, query, top_k=3):
        results = self.vector_store.query(query, top_k)
        return results['documents']

    def answer(self, question):
        relevant_docs = self.retrieve(question)
        context = "\n".join(relevant_docs)

        prompt = f"""基于以下信息回答问题。如果信息不足，请说明你不知道。

信息：
{context}

问题： {question}
答案： ""

        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=[{"role": "user": content, "content": prompt}]
        )

        return response.choices[0].message["content"]
```



RAG的优势： 1) 可以处理超长文档 2) 答案有据可查 3) 可以实时更新知识库 4) 节省token

4 RAG的局限

RAG虽好，但也不是万能的：

问题	原因	解决方案
检索不到相关内容	Embedding模型不匹配	用更优质的Embedding服务

问题	原因	解决方案
答案不连贯	检索到的片段太碎片化	增加上下文窗口大小
幻觉问题	模型过度发挥	在Prompt中强调"基于提供的信息"

说完向量数据库和RAG，接下来我们来做一个实战——让Agent记住你是谁。

实战：让Agent记住你是谁

学完了记忆系统的理论知识，这一期我们来做个实战——写一个能记住用户信息的Agent。

 **目标：** 构建一个完整的记忆系统，能跨对话记住用户的基本信息和偏好。

1 需求分析

我们要实现以下功能：

- 记住用户的姓名、联系方式、偏好等基本信息
- 跨对话保持记忆（重启程序后不丢失）
- 能根据记忆提供个性化服务

2 完整代码

PYTHON

```
import json
import os
```

```
class PersistentMemoryAgent:
    def __init__(self, memory_file="user_memory.json"):
        self.memory_file = memory_file
        self.memory = self.load_memory()
        self.messages = []

    def load_memory(self):
        if os.path.exists(self.memory_file):
            with open(self.memory_file, 'r') as f:
                return json.load(f)
        return {"user_info": {}, "preferences": {}, "history": []}

    def save_memory(self):
        with open(self.memory_file, 'w') as f:
            json.dump(self.memory, f, ensure_ascii=False, indent=2)

    def update_memory(self, key, value):
        if "user_info" in key:
            self.memory["user_info"][key] = value
        else:
            self.memory["preferences"][key] = value
        self.save_memory()

    def build_context(self):
        context = []

        user_info = self.memory.get("user_info", {})
        if user_info:
            info_text = "用户信息: "
            for k, v in user_info.items():
                info_text += f"\n- {k}: {v}"
            context.append({"role": "system", "content": info_text})

        context.extend(self.messages[-10:])
        return context

    def chat(self, user_input):
        self.messages.append({"role": "user", "content": user_input})

        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=self.build_context()
        )

        answer = response.choices[0].message["content"]
```

```
self.messages.append({"role": "assistant", "content": answer})  
return answer
```

3 使用示例

```
agent = PersistentMemoryAgent()  
  
agent.update_memory("user_info:name", "张三")  
agent.update_memory("user_info:city", "北京")  
agent.update_memory("preferences:coffee", "美式咖啡, 不加糖")  
  
print(agent.chat("附近有什么好咖啡店吗? "))  
# Agent会回答: 我知道你在北京, 喜欢美式咖啡不加糖...
```

💡 **关键点:** 1) 使用JSON文件持久化存储记忆 2) 每次构建上下文时注入用户信息 3) 对话历史限制在最近10条

到这里，记忆系统篇就全部讲完了。我们学了四层记忆架构、上下文窗口管理、向量数据库和RAG，最后还做了一个实战项目。

有问题欢迎留言。

📎 相关系列阅读

- OpenClaw原理篇12：记忆系统 {原文件:12-记忆系统-公众号版.md}
- AI Coding第28篇：Agent核心实现 {原文件:AI-28-Agent核心实现-公众号版.md}

下期预告：Agent开发系列 13-14 — Tool接口设计 + 内置工具实现

Agent开发系列 · 用代码理解AI

公众号：大前端工程师
公众号：大前端工程师
公众号：大前端工程师

Tool接口设计

之前我们说了记忆系统。Agent能记住东西了，下一步是让它能"做事"。

这一篇，我们来说说Tool接口设计——怎么让Agent调用工具。(参考[OpenClaw进阶篇09](#))

💡 **核心观点：** Agent能做什么事，取决于它有多少工具。好的Tool接口设计，让扩展工具变得简单。

1 为什么需要统一的接口

想象一下，如果没有统一的接口，每个工具都得单独写调用代码。搜索要写一套、计算器要写一套、发邮件又要写一套。

有了统一的Tool接口，所有工具都遵循同样的规范。注册、调用、返回结果，都是一套代码。

2 Tool基类

```
from abc import ABC, abstractmethod
from typing import Any, Dict

class BaseTool(ABC):
    """所有工具的基类"""

    def name(self) -> str:
        """工具名称，Agent通过这个名字调用"""
        raise NotImplementedError

    def description(self) -> str:
```

```

"""工具描述, 让模型知道什么时候该用"""
raise NotImplementedError

def parameters(self) -> Dict:
    """参数schema, 用于Function Calling"""
    raise NotImplementedError

def execute(self, **kwargs) -> Any:
    """实际执行工具逻辑"""
    pass

```

3 参数校验

参数校验很重要。不能让Agent传给工具的参数是乱七八糟的。

```

from pydantic import BaseModel, ValidationError

class WeatherInput(BaseModel):
    city: str
    date: str = "today"

class WeatherTool(BaseTool):
    @property
    def name(self): return "get_weather"

    @property
    def description(self):
        return "查询指定城市的天气"

    @property
    def parameters(self):
        return {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "城市名称"},
                "date": {"type": "string", "description": "日期, 默认为today"}
            },
            "required": ["city"]
        }

    def execute(self, city, date="today"):

```

```
try:
    WeatherInput(city=city, date=date)
except ValidationError:
    return "参数错误：城市名称不能为空"
return f"{city} {date}的天气：晴，25度"
```

4 结果格式化

工具返回的结果也要格式化，不然Agent可能看不懂。

```
class ToolResult:
    def __init__(self, success, data, error=None):
        self.success = success
        self.data = data
        self.error = error

    def __str__(self):
        if self.success:
            return f"成功: {self.data}"
        return f"失败: {self.error}"
```

💡 **Tool设计的要点：** 1) 统一接口方便扩展 2) 参数校验防止错误 3) 结果格式化让Agent好理解

内置工具实现

Tool接口设计好了，接下来看看常见的内置工具怎么实现。

💡 **核心观点：** 内置工具是Agent的"基本功"。搜索、计算、文件、代码——这些是每个Agent都应该会的东西。

1 搜索工具

```
PYTHON

class SearchTool(BaseTool):
    @property
    def name(self): return "search"

    @property
    def description(self):
        return "搜索互联网获取信息"

    @property
    def parameters(self):
        return {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "搜索关键词"}
            },
            "required": ["query"]
        }

    def execute(self, query):
        import requests
        response = requests.get(
            "https://api.search.com",
            params={"q": query}
        )
        return response.json()
```

2 计算工具

```
PYTHON

class CalculatorTool(BaseTool):
    @property
    def name(self): return "calculate"
```

```
@property
def description(self):
    return "执行数学计算"

@property
def parameters(self):
    return {
        "type": "object",
        "properties": {
            "expression": {"type": "string", "description": "数学表达式，如"},
        },
        "required": ["expression"]
    }

def execute(self, expression):
    try:
        result = eval(expression)
        return f"{expression} = {result}"
    except Exception as e:
        return f"计算错误: {e}"
```

3 文件工具

```
class FileTool(BaseTool):
    @property
    def name(self): return "read_file"

    @property
    def description(self):
        return "读取文件内容"

    @property
    def parameters(self):
        return {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "文件路径"}
            },
            "required": ["path"]
        }
```

```
def execute(self, path):
    import os
    if not os.path.exists(path):
        return f"文件不存在: {path}"
    with open(path, 'r') as f:
        return f.read()
```

4 工具注册中心

所有工具注册到中心，方便管理：

```
class ToolRegistry:
    def __init__(self):
        self.tools = {}

    def register(self, tool: BaseTool):
        self.tools[tool.name] = tool

    def get_tool(self, name):
        return self.tools.get(name)

    def list_tools(self):
        return [(t.name, t.description) for t in self.tools.values()]

    def get_tools_for_llm(self):
        return [
            {
                "type": "function",
                "function": {
                    "name": t.name,
                    "description": t.description,
                    "parameters": t.parameters
                }
            }
            for t in self.tools.values()
        ]
```



工具注册中心的好处： 1) 统一管理所有工具 2) 方便添加/删除工具 3) 一键生成

LLM可用的tools参数

有问题欢迎留言。

相关系列阅读

- OpenClaw进阶篇09：工具与API集成 {原文件:09-工具与API集成-公众号版.md}
- AI Coding第29篇：工具系统 {原文件:AI-29-工具系统-Tool-System-公众号版.md}

下期预告：Agent开发系列 15-16 — Function Calling + 工具安全

Agent开发系列 · 用代码理解AI

Function Calling

上一期我们说了Tool接口设计和内置工具实现。工具写好了，下一步是让Agent能"调用"它们。

这一篇，我们来说说Function Calling——LLM与工具之间的桥梁。

 **核心观点：** Function Calling让LLM能够识别何时需要调用工具，并正确传递参数。

1 什么是Function Calling

普通的LLM对话，模型只能生成文字。但有了Function Calling，模型可以告诉你："我需要调用某个函数来处理这个请求。"

比如用户说"帮我查一下上海天气"，模型识别出需要调用天气工具，返回的结构化数据包含：

- 函数名：get_weather
- 参数：{"city": "上海", "date": "today"}

2 OpenAI的Function Calling

```
PYTHON

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "帮我查一下上海天气"}],
    tools=[
        {
            "type": "function",
```

```

        "function": {
            "name": "get_weather",
            "description": "查询城市天气",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "城市名"}
                },
                "required": ["city"]
            }
        }
    }
]
)

# response.choices[0].message 会包含 tool_calls

```

3 完整调用流程

```

class AgentWithFunctionCalling:
    def __init__(self, tools):
        self.tools = tools
        self.messages = []

    def chat(self, user_input):
        self.messages.append({"role": "user", "content": user_input})

        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=self.messages,
            tools=self.tools,
            tool_choice="auto"
        )

        message = response.choices[0].message

        if message.get("tool_calls"):
            return self.handle_tool_call(message)
        else:
            self.messages.append(message)
            return message["content"]

```

```
def handle_tool_call(self, message):
    tool_calls = message["tool_calls"]
    for call in tool_calls:
        tool_name = call["function"]["name"]
        args = json.loads(call["function"]["arguments"])

        result = self.execute_tool(tool_name, args)

        self.messages.append({
            "role": "tool",
            "tool_call_id": call["id"],
            "content": str(result)
        })

    return self.continue_chat()
```

4 提示词设计技巧

Function Calling的效果，很大程度上取决于提示词的设计。

技巧	做法	效果
清晰的工具描述	描述工具能做什么，而不是怎么做	减少误调用
具体参数描述	说明每个参数的意义和格式	参数更准确
when to use	说明什么时候该用这个工具	减少不必要的调用

💡 **提示词示例：** "当用户询问实时信息（天气、股价、新闻）时，调用搜索工具。当用户需要计算时，调用计算器工具。"

工具安全

Function Calling让Agent能调用工具了。但工具调用也带来了安全问题——Agent可能会执行危险操作。

💡 **核心观点：** 工具是Agent的"手"，必须确保这双手不会伤害用户或系统。

1 权限控制

不是所有用户都能调用所有工具。要根据用户身份限制权限。

```
PYTHON

class PermissionChecker:
    def __init__(self):
        self.permissions = {
            "admin": ["*"], # 所有工具
            "user": ["search", "calculate", "read_file"],
            "guest": ["search"]
        }

    def can_use(self, user_role, tool_name):
        allowed = self.permissions.get(user_role, [])
        return "*" in allowed or tool_name in allowed
```

2 沙箱隔离

危险工具要在隔离环境中执行，防止对系统造成伤害。

```
PYTHON

class SandboxedExecutor:
    def execute_code(self, code):
        import subprocess
        import tempfile

        with tempfile.NamedTemporaryFile(mode='w', suffix='.py') as f:
            f.write(code)
```

```
f.flush()
result = subprocess.run(
    ["python", f.name],
    capture_output=True,
    timeout=5, # 超时5秒
    cwd="/tmp" # 限制工作目录
)
return result.stdout.decode()
```

3 危险操作拦截

一些高危操作需要额外确认。

```
class DangerousOperationGuard:
    DANGEROUS_PATTERNS = [
        "rm -rf",
        "drop table",
        "format disk",
        "exec(",
    ]

    def check(self, operation):
        for pattern in self.DANGEROUS_PATTERNS:
            if pattern in operation.lower():
                return False, f"检测到危险操作: {pattern}"
        return True, "安全"
```

⚠ **安全要点:** 1) 权限控制按需分配 2) 危险操作要拦截 3) 代码执行要沙箱隔离 4) 所有操作要留日志

有问题欢迎留言。

 相关系列阅读

- OpenClaw进阶篇09：工具与API集成 {原文件:09-工具与API集成-公众号版.md}
- AI Coding第29篇：工具系统 {原文件:AI-29-工具系统-Tool-System-公众号版.md}

下期预告：Agent开发系列 17-18 — 任务分解 + 执行引擎

Agent开发系列 · 用代码理解AI

任务分解

之前我们说了工具系统。Agent能调用工具了，下一步是让它能处理复杂任务。

复杂任务不能一步完成，需要分解成多个子任务。这一篇，我们来说说任务分解。

💡 **核心观点：** 复杂任务 = 多个简单任务的组合。任务分解让Agent能够处理超出单次调用能力范围的工作。

1 为什么要任务分解

想象用户说："帮我写一篇关于AI的文章，包括标题、正文，还要配上封面图。"

这个任务至少包含三个子任务：

1. 写标题
2. 写正文
3. 生成封面图

如果Agent不分解任务，它可能会：

- 只写了标题就结束了
- 标题和正文混在一起，结构混乱
- 忘记配图

2 自动任务分解



PYTHON

```
class TaskDecomposer:
    def decompose(self, task):
        prompt = f"""将以下任务分解成子任务:

任务: {task}

请将任务分解成独立的子任务, 每个子任务应该:
1. 可以独立执行
2. 有明确的完成标准
3. 不依赖其他子任务的结果 (如果依赖请标注)

以JSON数组格式返回:
[{{"id": 1, "task": "子任务描述", "depends_on": []}}]"""

        response = openai.ChatCompletion.create(
            model="gpt-4",
            messages=[{"role": "user", "content": prompt}]
        )

        import json
        return json.loads(response.choices[0].message["content"])
```

3 依赖分析与并行规划

分解后的子任务, 有些可以并行执行, 有些必须按顺序执行。

```
class TaskScheduler:
    def create_execution_plan(self, tasks):
        # 构建依赖图
        graph = {t["id"]: t.get("depends_on", []) for t in tasks}

        # 找出可以并行的任务
        plan = []
        completed = set()

        while len(completed) < len(tasks):
            # 找出所有依赖都已完成的任务
            ready = []
            for task in tasks:
                tid = task["id"]
```

```
        if tid in completed:
            continue
        if all(dep in completed for dep in graph[tid]):
            ready.append(task)

    if not ready:
        break # 环形依赖

    plan.append(ready)
    for t in ready:
        completed.add(t["id"])

    return plan # [[task1, task2], [task3]] 表示可并行, 执行顺序
```

💡 **任务分解的好处:** 1) 处理复杂任务 2) 任务可并行执行提高效率 3) 便于追踪进度和错误定位

执行引擎

任务分解完了, 接下来是执行。执行引擎负责按照计划执行每个子任务。

💡 **核心观点:** 执行引擎就是任务的"调度中心", 管理任务的执行顺序、状态和结果。

1 状态机设计

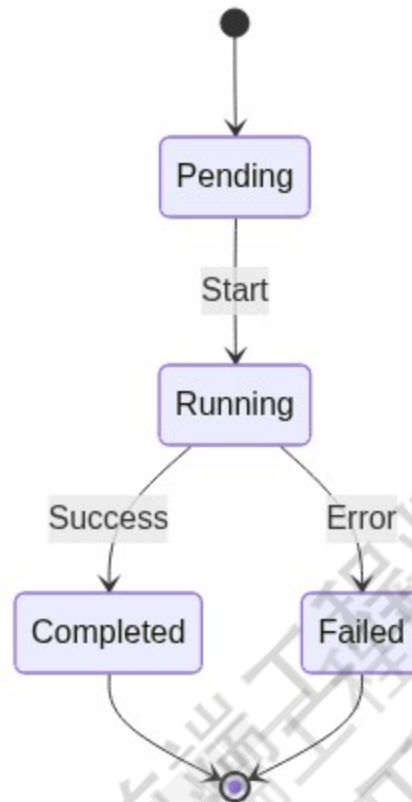


Figure: Task State Machine - Pending -> Running -> Completed/Failed

每个任务都有生命周期：pending → running → completed/failed

```
PYTHON

class Task:
    def __init__(self, task_id, description):
        self.task_id = task_id
        self.description = description
        self.status = "pending" # pending, running, completed, failed
        self.result = None
        self.error = None
```

2 任务队列

```
PYTHON

class ExecutionEngine:
    def __init__(self, agent):
```

```

self.agent = agent
self.tasks = {}
self.results = {}

def execute_plan(self, plan):
    for batch in plan:
        # 同一批的任务可以并行执行
        batch_results = []
        for task_spec in batch:
            result = self.execute_task(task_spec)
            batch_results.append(result)

        for result in batch_results:
            self.results[result["task_id"]] = result

    return self.results

def execute_task(self, task_spec):
    task_id = task_spec["id"]
    task_desc = task_spec["task"]

    try:
        result = self.agent.run(task_desc)
        return {
            "task_id": task_id,
            "status": "completed",
            "result": result
        }
    except Exception as e:
        return {
            "task_id": task_id,
            "status": "failed",
            "error": str(e)
        }

```

3 进度跟踪

```

class ProgressTracker:
    def get_progress(self, results):
        total = len(results)
        completed = sum(1 for r in results.values() if r.get("status") == "completed")
        failed = sum(1 for r in results.values() if r.get("status") == "failed")

```

```
return {  
    "total": total,  
    "completed": completed,  
    "failed": failed,  
    "percentage": int(completed / total * 100) if total > 0 else 0  
}
```

💡 **执行引擎的核心功能：** 1) 按计划执行任务 2) 管理任务状态 3) 收集和存储结果 4) 跟踪执行进度

有问题欢迎留言。

📎 相关系列阅读

- OpenClaw原理篇11：核心循环 {原文件:11-核心循环-公众号版.md}

下期预告：Agent开发系列 19-20 — 动态调整 + 协作模式设计

Agent开发系列 · 用代码理解AI

动态调整

任务分解和执行引擎讲完了。但在实际执行中，计划往往赶不上变化。子任务可能失败、外部条件可能改变、用户需求可能调整——这时候就需要动态调整。

💡 **核心观点：**好的执行系统不是一条道走到黑，而是能根据实际情况灵活应变。动态调整让Agent系统更健壮。

1 回滚机制

当某个子任务失败时，需要回滚已完成的操作，保持系统状态一致。

```
PYTHON

class RollbackManager:
    def __init__(self):
        self.history = []

    def save_state(self, state):
        self.history.append(state)

    def rollback(self):
        if not self.history:
            return None
        state = self.history.pop()
        return state

    def handle_failure(self, failed_task, context):
        # 回滚到上一个稳定状态
        previous_state = self.rollback()
        if previous_state:
            context.restore(previous_state)
        return {
            "action": "replan",
```

```
"reason": f"Task {failed_task} failed"
}
```

2 降级策略

当理想方案不可用时，自动切换到备选方案。

```
class FallbackStrategy:
    def __init__(self):
        self.fallbacks = {
            "image_gen": ["dalle", "stable_diffusion", "placeholder"],
            "search": ["google", "bing", "cache"],
            "llm": ["gpt-4", "gpt-3.5", "local"]
        }

    def execute_with_fallback(self, capability, func, *args):
        fallbacks = self.fallbacks.get(capability, [])
        for fallback in fallbacks:
            try:
                result = func(fallback, *args)
                return result
            except Exception as e:
                continue
        raise Exception(f"All fallbacks failed for {capability}")
```

3 用户干预

```
class DynamicExecutor:
    def execute_with_intervention(self, plan):
        for batch in plan:
            for task in batch:
                result = self.check_and_execute(task)
                if result.get("needs_intervention"):
                    user_choice = self.request_user_input(result)
                    if user_choice == "abort":
                        return {"status": "aborted"}
```

```
elif user_choice == "modify":
    modified_plan = self.replan(result)
    return self.execute_with_intervention(modified_plan)

def check_and_execute(self, task):
    # 高风险操作需要用户确认
    if task.get("risk_level") > 0.7:
        return {"needs_intervention": True, "task": task}
    return self.execute_task(task)
```

💡 **动态调整的三板斧：**回滚恢复状态、降级切换方案、干预交给用户。这三招让系统面对意外时也能稳如泰山。

协作模式设计

单个Agent能力有限，遇到复杂任务时需要多个Agent协作。这一篇，我们来看多Agent协作的基本模式。

💡 **核心观点：**多Agent协作不是简单地把多个Agent串起来，而是要设计好协作模式，让它们各司其职、高效配合。

1

三种协作模式

模式	特点	适用场景
流水线	顺序执行，上游输出是下游输入	内容生产、数据处理
主从	一个主Agent协调多个从Agent	复杂任务分解、团队管理
对等	所有Agent平等协作，共享目标	头脑风暴、方案评审

2 流水线模式

```
PYTHON

class PipelineOrchestrator:
    def __init__(self):
        self.stages = []

    def add_stage(self, name, agent):
        self.stages.append({"name": name, "agent": agent})

    def execute(self, initial_input):
        context = {"input": initial_input}
        for stage in self.stages:
            agent = stage["agent"]
            result = agent.run(context)
            context[stage["name"]] = result
            context["input"] = result # 传递给下一个阶段
        return context

pipeline = PipelineOrchestrator()
pipeline.add_stage("research", research_agent)
pipeline.add_stage("write", write_agent)
pipeline.add_stage("review", review_agent)
result = pipeline.execute("AI发展趋势")
```

3 主从模式

```
PYTHON

class MasterSlaveOrchestrator:
    def __init__(self, master_agent):
        self.master = master_agent
        self.slaves = {}

    def register_slave(self, name, agent, capabilities):
        self.slaves[name] = {
            "agent": agent,
            "capabilities": capabilities
        }

    def execute(self, task):
        subtasks = self.master.decompose(task)
```

```
results = {}
for subtask in subtasks:
    slave_name = self.master.assign(subtask, self.slaves)
    slave = self.slaves[slave_name]["agent"]
    results[subtask["id"]] = slave.run(subtask)
return self.master.aggregate(results)
```

4 对等模式

```
class PeerToPeerOrchestrator:
    def __init__(self):
        self.peers = []
        self.message_bus = MessageBus()

    def add_peer(self, agent):
        self.peers.append(agent)
        agent.set_message_bus(self.message_bus)

    def execute(self, goal):
        # 每个peer独立工作，通过message bus共享进展
        threads = []
        for peer in self.peers:
            t = Thread(target=peer.work, args=(goal,))
            threads.append(t)
            t.start()
        for t in threads:
            t.join()
        return self.message_bus.get_all_results()
```

💡 **协作模式选择指南：** 流水线适合线性流程、主从适合任务分配、对等适合并行探索。根据场景选对模式，协作效率翻倍。

有问题欢迎留言。

相关系列阅读

- Agent开发系列 17-18：任务分解 + 执行引擎

下期预告：Agent开发系列 21-22 — 消息总线 + 团队编排

Agent开发系列 · 用代码理解AI

公众号：大前端工程师
公众号：大前端工程师
公众号：大前端工程师

消息总线

多Agent协作需要一个通信中枢。消息总线就是扮演这个角色——它让Agent之间的消息传递变得可靠、可追踪。

💡 **核心观点：**消息总线是Agent系统的"神经系统"，负责消息路由、事件分发和状态同步。

1 消息类型

```
from enum import Enum

class MessageType(Enum):
    TASK = "task"           # 任务分配
    RESULT = "result"       # 结果返回
    EVENT = "event"         # 事件通知
    QUERY = "query"         # 状态查询
    BROADCAST = "broadcast" # 广播消息

class Message:
    def __init__(self, msg_type, sender, content, target=None):
        self.type = msg_type
        self.sender = sender
        self.target = target
        self.content = content
        self.timestamp = datetime.now()
        self.id = uuid.uuid4().hex()
```

2 消息总线实现

```

class MessageBus:
    def __init__(self):
        self.subscribers = {}
        self.message_queue = Queue()
        self.message_history = []

    def subscribe(self, agent, msg_types):
        for msg_type in msg_types:
            if msg_type not in self.subscribers:
                self.subscribers[msg_type] = []
            self.subscribers[msg_type].append(agent)

    def publish(self, message):
        self.message_history.append(message)
        self.message_queue.put(message)

    def dispatch(self):
        while not self.message_queue.empty():
            message = self.message_queue.get()
            targets = self.subscribers.get(message.type, [])
            for target in targets:
                target.receive(message)

    def send(self, from_agent, to_agent, content):
        msg = Message(MessageType.TASK, from_agent, content, to_agent)
        self.publish(msg)

```

3 事件分发

```

class EventBus:
    # 事件类型
    def on_task_complete(self, task_id):
        return {
            "event": "task_complete",
            "task_id": task_id
        }

    def on_agent_error(self, agent_id, error):
        return {
            "event": "agent_error",
            "agent_id": agent_id,

```

```

        "error": error
    }

    def emit(self, event_data):
        self.message_bus.publish(
            Message(MessageType.EVENT, "system", event_data)
        )

```

💡 **消息总线的核心能力：** 订阅发布机制实现松耦合、消息队列保证异步通信、历史记录便于调试追踪。

团队编排

多个Agent一起工作，需要有人来分配任务、协调进度、汇总结果。这就是团队编排。

💡 **核心观点：** 团队编排解决的是"谁来做、做什么、结果怎么合"三个问题。

1 角色定义

```

class AgentRole:
    PLANNER = "planner"      # 规划者：分解任务
    EXECUTOR = "executor"    # 执行者：完成任务
    COORDINATOR = "coordinator" # 协调者：分配任务
    REVIEWER = "reviewer"    # 审核者：质量把控

class TeamMember:
    def __init__(self, agent, role, capabilities):
        self.agent = agent
        self.role = role
        self.capabilities = capabilities
        self.current_task = None

```

2 任务调度器

```
PYTHON

class TeamOrchestrator:
    def __init__(self):
        self.members = []
        self.task_queue = Queue()
        self.results = {}

    def add_member(self, agent, role, capabilities):
        member = TeamMember(agent, role, capabilities)
        self.members.append(member)
        return member

    def assign_task(self, task, member):
        member.current_task = task
        result = member.agent.execute(task)
        self.results[task["id"]] = result
        member.current_task = None
        return result

    def find_best_member(self, task):
        required_caps = task.get("capabilities", [])
        best = None
        best_score = 0
        for member in self.members:
            score = sum(1 for cap in required_caps if cap in member.capabilities)
            if score > best_score:
                best_score = score
                best = member
        return best
```

3 结果汇总

```
PYTHON

class ResultAggregator:
    def aggregate(self, results, strategy="merge"):
        if strategy == "merge":
            return self.merge_results(results)
        elif strategy == "chain":
            return self.chain_results(results)
```

```
elif strategy == "vote":
    return self.vote_results(results)

def merge_results(self, results):
    merged = {}
    for task_id, result in results.items():
        merged.update(result)
    return merged

def chain_results(self, results):
    ordered = sorted(results.items(), key=lambda x: x[0])
    output = ""
    for _, result in ordered:
        output += result.get("content", "")
    return {"content": output}

def vote_results(self, results):
    votes = Counter()
    for result in results.values():
        answer = result.get("answer")
        votes[answer] += 1
    return {"answer": votes.most_common(1)[0][0]}
```

💡 **团队编排的关键：**明确角色分工、智能任务分配、合理结果汇总。三者缺一不可。

有问题欢迎留言。

📎 相关系列阅读

- Agent开发系列 19-20：动态调整 + 协作模式设计

下期预告：Agent开发系列 23-24 — 内容生产团队 + 性能优化

Agent开发系列 · 用代码理解AI

内容生产团队

这一篇，我们用一个实战案例把多Agent协作串起来：做一个内容生产团队，包括策划、写作、编辑、发布四个Agent。

💡 **核心观点：**多Agent系统的威力在于专业分工。每个Agent专注自己的领域，通过协作完成复杂任务。

1 团队架构

```
PYTHON

class ContentProductionTeam:
    def __init__(self):
        self.message_bus = MessageBus()
        self.orchestrator = TeamOrchestrator()

        # 创建各个Agent
        self.planner = PlannerAgent()
        self.writer = WriterAgent()
        self.editor = EditorAgent()
        self.publisher = PublisherAgent()

        # 注册团队成员
        self.orchestrator.add_member(self.planner, AgentRole.PLANNER, ["planning"])
        self.orchestrator.add_member(self.writer, AgentRole.EXECUTOR, ["writing"])
        self.orchestrator.add_member(self.editor, AgentRole.REVIEWER, ["editing"])
        self.orchestrator.add_member(self.publisher, AgentRole.COORDINATOR, ["publishing"])

    def produce_content(self, topic):
        # 第一步：策划
        plan = self.orchestrator.assign_task(
            {"id": "1", "task": f"策划一篇关于{topic}的文章", "capabilities": ["planning"]},
            self.planner_member
        )

        # 第二步：写作
```

```

draft = self.orchestrator.assign_task(
    {"id": "2", "task": "根据策划写文章草稿", "capabilities": ["writing"]}
    self.writer_member
)

# 第三步：编辑
edited = self.orchestrator.assign_task(
    {"id": "3", "task": "编辑审核文章", "capabilities": ["editing"]},
    self.editor_member
)

# 第四步：发布
result = self.orchestrator.assign_task(
    {"id": "4", "task": "发布文章到平台", "capabilities": ["publishing"]}
    self.publisher_member
)

return result

```

2 Agent实现

```

class PlannerAgent:
    def execute(self, task):
        prompt = f"""为文章制定写作计划：

任务：{task['task']}

请制定：
1. 文章主题和角度
2. 核心观点（3-5个）
3. 文章结构大纲
4. 需要的研究资料"""

        response = llm.generate(prompt)
        return {"plan": response, "status": "completed"}

class WriterAgent:
    def execute(self, task):
        plan = task.get("plan")
        prompt = f"根据以下计划撰写文章：\n\n{plan}"
        draft = llm.generate(prompt)
        return {"draft": draft, "status": "completed"}

```

```
class EditorAgent:
    def execute(self, task):
        draft = task.get("draft")
        prompt = f"""编辑审核以下文章：

{draft}

请检查并修正：
1. 语法错误
2. 逻辑问题
3. 表达不清的地方
4. 格式问题"""

        edited = llm.generate(prompt)
        return {"edited": edited, "status": "completed"}

class PublisherAgent:
    def execute(self, task):
        article = task.get("edited")
        platform = self.select_platform(article)
        result = self.publish(article, platform)
        return {"published": result, "platform": platform}
```

💡 **内容生产团队的优势：**专业分工、流水线作业、质量把控。一个需求进去，完整内容出来。

性能优化

代码跑通了，接下来要考虑性能。Agent系统常见的性能问题有哪些？怎么优化？

💡 **核心观点：**性能优化三个方向：减少等待、并行处理、合理缓存。抓住这三点，Agent系统效率翻倍。

1 并发处理

```
PYTHON

import asyncio
from concurrent.futures import ThreadPoolExecutor

class AsyncExecutionEngine:
    def __init__(self, max_workers=10):
        self.executor = ThreadPoolExecutor(max_workers=max_workers)

    async def execute_parallel(self, tasks):
        loop = asyncio.get_event_loop()
        futures = [
            loop.run_in_executor(self.executor, task)
            for task in tasks
        ]
        results = await asyncio.gather(*futures)
        return results

    async def execute_batch(self, batch):
        # 同一批次的任务可以并行执行
        return await self.execute_parallel(batch)
```

2 缓存策略

```
PYTHON

from functools import lru_cache
import hashlib

class ResultCache:
    def __init__(self, max_size=1000):
        self.cache = {}
        self.max_size = max_size

    def _make_key(self, task):
        # 基于任务内容生成缓存key
        content = str(task.get("task")) + str(task.get("context"))
        return hashlib.md5(content.encode()).hexdigest()

    def get(self, task):
        key = self._make_key(task)
```

```
return self.cache.get(key)

def set(self, task, result):
    if len(self.cache) >= self.max_size:
        # LRU淘汰
        self.cache.pop(next(iter(self.cache)))
    key = self._make_key(task)
    self.cache[key] = result
```

3 流式输出

```
PYTHON

async def stream_generate(prompt):
    # 使用流式API减少等待时间
    stream = await openai.ChatCompletion.acreate(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}],
        stream=True
    )

    accumulated = ""
    async for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            accumulated += delta
            yield delta # 实时yield, 每个token
```

💡 **性能优化 Checklist:** 1) 能并行的任务不串行 2) 重复计算要缓存 3) 流式输出减少等待 4) 合理设置超时

有问题欢迎留言。

📎 相关系列阅读

- Agent开发系列 21-22：消息总线 + 团队编排

下期预告：Agent开发系列 25-26 — 可观测性 + 生产部署

Agent开发系列 · 用代码理解AI

公众号：大前端工程师
公众号：大前端工程师
公众号：大前端工程师

可观测性

Agent系统上线后，怎么知道它运行得好不好？有没有出问题？

传统软件出了问题，可以看日志、查数据库。但Agent系统复杂在哪？它涉及多次LLM调用、工具执行、多步骤推理——一个请求可能跨十几个服务、产生上百条日志。

可观测性就是回答这些问题的：[我的Agent在想什么？它为什么这么做？哪里出了问题？](#)

核心观点：可观测性三板斧——[日志（Log）](#)记录事件、[指标（Metrics）](#)反映趋势、[追踪（Trace）](#)串联调用链。做好这三点，生产环境的问题无处遁形。

1 日志设计：让Agent的每一步都有迹可循

日志是排查问题的第一手资料。但Agent的日志和普通系统不同——我们需要记录的不仅是“系统发生了什么”，还要记录“[Agent在想什么](#)”。

看这段日志设计：

```
PYTHON

import logging
import json
from datetime import datetime

class AgentLogger:
    def __init__(self, agent_id):
        self.agent_id = agent_id
        self.logger = logging.getLogger(f"agent.{agent_id}")

    def log(self, level, event, data=None):
        log_entry = {
            "timestamp": datetime.now().isoformat(),
```

```
        "agent_id": self.agent_id,
        "event": event,
        "data": data or {}
    }
    if level == "error":
        self.logger.error(json.dumps(log_entry))
    else:
        self.logger.info(json.dumps(log_entry))

    def log_task_start(self, task):
        self.log("info", "task_start", {"task": task})

    def log_llm_call(self, prompt, response, latency):
        self.log("info", "llm_call", {
            "prompt_tokens": len(prompt),
            "response_tokens": len(response),
            "latency_ms": latency
        })

    def log_tool_execution(self, tool_name, input_data, output, success):
        self.log("info", "tool_execution", {
            "tool": tool_name,
            "input": input_data,
            "output": output,
            "success": success
        })

    def log_task_complete(self, task_id, result, duration):
        self.log("info", "task_complete", {
            "task_id": task_id,
            "result": result,
            "duration_ms": duration
        })

    def log_error(self, error, context):
        self.log("error", "error", {
            "error": str(error),
            "context": context
        })
```

为什么这样设计？

- **结构化JSON**：方便后续用ELK、Graylog等工具搜索和分析
- **关键事件全覆盖**：任务开始、LLM调用、工具执行、任务完成、错误

- [上下文完整](#)：每条日志都带timestamp和agent_id，方便串联

2

指标采集：让趋势一目了然

日志告诉你"发生了什么"，但如果你想知道"最近怎么样"、"有没有异常趋势"，就需要指标了。

指标分三种类型：

类型	说明	举例
Counter（计数器）	只增不减，记录累计数量	总请求数、任务完成数
Gauge（仪表）	可增可减，记录瞬时值	当前活跃任务数、队列长度
Histogram（直方图）	记录分布情况	响应时间分布、Token消耗分布

PYTHON

```
class MetricsCollector:
    def __init__(self):
        self.counters = {}
        self.gauges = {}
        self.histograms = {}

    def increment(self, name, value=1):
        self.counters[name] = self.counters.get(name, 0) + value

    def gauge(self, name, value):
        self.gauges[name] = value

    def histogram(self, name, value):
        if name not in self.histograms:
            self.histograms[name] = []
        self.histograms[name].append(value)

    def get_metrics(self):
        return {
            "counters": self.counters,
            "gauges": self.gauges,
```

```

        "histograms": {
            k: {"count": len(v),
               "avg": sum(v)/len(v) if v else 0
            }
            for k, v in self.histograms.items()
        }
    }

    def export_to_prometheus(self):
        lines = []
        for name, value in self.counters.items():
            lines.append(f"agent_counter{name=\"{name}\"} {value}")
        for name, value in self.gauges.items():
            lines.append(f"agent_gauge{name=\"{name}\"} {value}")
        return "\n".join(lines)

# 使用示例
metrics = MetricsCollector()
metrics.increment("task_completed")
metrics.histogram("task_duration", 1.5) # 秒
metrics.histogram("token_usage", 2048)
metrics.gauge("active_tasks", 5)

```

Agent系统关键指标有哪些？

- **任务级指标**：任务成功率、平均耗时、超时率
- **LLM级指标**：Token消耗、平均延迟、错误率
- **工具级指标**：各工具调用次数、成功率和平均耗时
- **资源级指标**：活跃任务数、队列长度、内存使用

3 分布式追踪：串联Agent的思维链

这是Agent可观测性最有特色的部分。

普通API调用可以用Trace ID串联。但Agent不同——它有**思维链（Chain of Thought）**，每个思考步骤可能涉及多次LLM调用和工具执行。

我们需要追踪的是：**从用户输入到最终输出，Agent经历了什么思考过程？**

```
from contextvars import ContextVar
import time
import uuid

# 追踪上下文 - 自动在协程间传递
trace_id: ContextVar[str] = ContextVar("trace_id", default="")
span_id: ContextVar[str] = ContextVar("span_id", default="")

class DistributedTracer:
    def create_trace(self, user_input):
        new_trace_id = uuid.uuid4().hex[:16]
        trace_id.set(new_trace_id)
        return {
            "trace_id": new_trace_id,
            "user_input": user_input,
            "start_time": time.time()
        }

    def start_span(self, operation_name, parent_span_id=None):
        span_id_val = uuid.uuid4().hex[:8]
        current_span_id = span_id.get()
        span.set(span_id_val)
        return {
            "trace_id": trace_id.get(),
            "parent_span_id": parent_span_id or current_span_id,
            "span_id": span_id_val,
            "operation": operation_name,
            "start_time": time.time()
        }

    def end_span(self, span, status="ok", result=None):
        span["duration_ms"] = (time.time() - span["start_time"]) * 1000
        span["status"] = status
        if result:
            span["result"] = result
        self.report(span)
        return span

    def report(self, span):
        # 上报追踪数据到Jaeger/Zipkin/OpenTelemetry
        pass

tracer = DistributedTracer()
```

```
# 使用示例
trace = tracer.create_trace("帮我查一下北京天气")
span = tracer.start_span("llm.think")
think_result = do_think()
tracer.end_span(span, result=think_result)
```

可观测性实践：结构化日志便于搜索、指标帮助发现趋势、追踪串联请求全流程。三者结合，问题定位事半功倍。

生产部署

代码本地跑通了，下一步是上线生产环境。

这一篇说说Docker容器化、K8s编排和灰度发布——这三位一体，构成了现代Agent系统生产部署的标准打法。

核心观点：生产部署三件套——**容器化**保证环境一致、**K8s**保证弹性伸缩、**灰度发布**保证平滑迭代。

1 Docker容器化：一次构建，处处运行

Agent系统依赖复杂——Python环境、LLM SDK、各种工具库。本地跑通不代表生产能跑。

Docker容器把代码和依赖打包在一起，保证**构建环境 = 运行环境**。

```
# Dockerfile
FROM python:3.11-slim
```

DOCKERFILE

```
# 安装系统依赖
RUN apt-get update && apt-get install -y \
    git curl && \
    rm -rf /var/lib/apt/lists/*

# 安装Python依赖
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 复制代码
COPY . /app
WORKDIR /app

# 环境变量
ENV PYTHONUNBUFFERED=1
ENV LOG_LEVEL=INFO
ENV OPENAI_API_KEY=${OPENAI_API_KEY}

# 健康检查 - 检查API是否正常
HEALTHCHECK --interval=30s --timeout=10s \
    --start-period=40s --retries=3 \
    CMD python health_check.py || exit 1

# 切换到非root用户运行
RUN useradd -m agent && chown -R agent:agent /app
USER agent

# 启动命令
CMD ["python", "main.py"]
```

关键配置解读：

- **python:3.11-slim**：比完整镜像小很多，减少攻击面
- **PYTHONUNBUFFERED=1**：日志实时输出，不用等缓冲区
- **HEALTHCHECK**：K8s用它判断容器是否存活
- **USER agent**：不以root运行，安全

2 K8s编排：弹性伸缩，故障自愈

单机跑Docker够了，但生产环境需要：多副本保证可用、自动扩容应对流量、滚动更新不中断服务。

Kubernetes就是干这个的。



```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agent-system
spec:
  replicas: 3
  selector:
    matchLabels:
      app: agent
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: agent
        version: v1
    spec:
      containers:
      - name: agent
        image: agent-system:1.0.0
        ports:
        - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        env:
        - name: OPENAI_API_KEY
          valueFrom:
            secretKeyRef:
```

```
      name: agent-secrets
      key: openai-api-key
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 15
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
```

YAML

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: agent-service
spec:
  selector:
    app: agent
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
```

关键概念解读：

配置	作用	为什么重要
replicas: 3	保持3个Pod副本	1个挂了还有2个，服务不断
resources	限制CPU和内存	防止单个容器占满节点
livenessProbe	判断容器是否要重启	程序死锁时自动恢复
readinessProbe	判断容器是否接收流量	启动中或有问题时不接流量

RollingUpdate

滚动更新策略

新版本慢慢替换，旧版本撑着

3 灰度发布：把风险装在笼子里

即使经过了测试，生产环境还是可能出问题。灰度发布就是让新版本**先让一小部分用户用**，没问题再全量。

```
# canary-deployment.yaml - 使用 Argo Rollouts 实现更精细的灰度
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: agent-rollout
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - 10%           # 先给10%的用户
        - pause: {}     # 暂停，观察指标
        - 30%           # 30%，继续观察
        - pause: {duration: 10m} # 等10分钟
        - 50%           # 50%
        - pause: {duration: 10m}
        - 100%          # 全量
      analysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: agent-service
  selector:
    matchLabels:
      app: agent
  template:
    metadata:
      labels:
        app: agent
    spec:
      containers:
        - name: agent
```

```
image: agent-system:1.1.0
ports:
- containerPort: 8080
```

灰度发布流程：

1. **10%流量**：先让10%用户访问新版本，观察错误率
2. **暂停观察**：如果指标正常，继续升级
3. **30%、50%、100%**：逐步放大，每次都留观察时间
4. **自动回滚**：如果错误率飙升，立即停止并回滚

部署 Checklist：

1. 容器化保证环境一致
2. 资源配置合理（不能光写不限制）
3. 健康检查不可少
4. 灰度发布控制风险
5. 监控告警要及时

有问题欢迎留言。

相关系列阅读

- Agent开发系列 23-24：内容生产团队 + 性能优化

下期预告：Agent开发系列 27-28 — 实战01 + 实战02

实战01：个人助手Agent

前面讲了这么多理论，这一篇开始实战。从零构建一个完整的个人助手Agent，功能包括：日程管理、信息查询、任务提醒。

💡 **核心观点：** 实战项目的价值在于把知识点串联起来。做一遍，比看十遍都有用。

1 项目结构

```
personal-assistant/  
├─ main.py           # 入口文件  
├─ agent/  
│   ├─ __init__.py  
│   ├─ core.py       # Agent核心  
│   ├─ memory.py     # 记忆系统  
│   └─ tools.py      # 工具定义  
├─ services/  
│   ├─ calendar.py   # 日历服务  
│   ├─ reminder.py   # 提醒服务  
│   └─ search.py     # 搜索服务  
└─ config.py         # 配置文件
```

2 Agent核心实现

```
# agent/core.py  
import os  
from typing import List, Dict, Any  
  
class PersonalAssistantAgent:  
    def __init__(self):  
        self.memory = ConversationMemory()
```

```

self.tools = ToolRegistry()
self.llm = OpenAI(model="gpt-4", api_key=os.getenv("OPENAI_API_KEY"))
self._register_tools()

def _register_tools(self):
    self.tools.register(CalendarTool())
    self.tools.register(ReminderTool())
    self.tools.register(SearchTool())

def process(self, user_input: str) -> str:
    context = self.memory.get_context()
    available_tools = self.tools.get_tools_for_llm()

    response = self.llm.chat(
        messages=[
            {"role": "system", "content": "你是一个贴心的个人助手..."},
            {"role": "user", "content": user_input}
        ],
        tools=available_tools
    )

    self.memory.add_message("user", user_input)
    self.memory.add_message("assistant", response.content)
    return response.content

```

3 工具实现

```

# agent/tools.py
from abc import ABC, abstractmethod
from typing import Dict, Any

class BaseTool(ABC):
    @property
    abstract def name(self) -> str: pass
    @property
    abstract def description(self) -> str: pass
    @abstractmethod
    def execute(self, **kwargs) -> Any: pass

class CalendarTool(BaseTool):
    @property
    def name(self): return "calendar"

```

```
@property
def description(self): return "查询或添加日历事件"
@property
def parameters(self):
    return {
        "action": {"type": "string", "enum": ["query", "add"]},
        "date": {"type": "string"},
        "event": {"type": "string"}
    }
def execute(self, action, date, event=None):
    if action == "query":
        return calendar_service.get_events(date)
    elif action == "add":
        return calendar_service.add_event(date, event)
```

💡 **实战要点：**模块化设计让代码清晰、工具注册机制让扩展容易、记忆系统让Agent变得"聪明"。

实战02：多Agent客服系统

第二个实战项目：构建一个多Agent客服系统。包含接待Agent、客服Agent、售后Agent，通过消息总线协作。

💡 **核心观点：**多Agent系统的核心是协调机制。消息总线、任务分发、结果聚合——这三件事做好，多Agent系统就跑起来了。

1 系统架构

```
# customer_service_system.py
class CustomerServiceSystem:
    def __init__(self):
```

PYTHON

```

self.message_bus = MessageBus()
self.orchestrator = TeamOrchestrator()

# 初始化三个Agent
self.reception = ReceptionAgent(self.message_bus)
self.support = SupportAgent(self.message_bus)
self.after_sales = AfterSalesAgent(self.message_bus)

# 注册到编排器
self.orchestrator.add_member(
    self.reception, AgentRole.COORDINATOR, ["reception"]
)
self.orchestrator.add_member(
    self.support, AgentRole.EXECUTOR, ["support"]
)
self.orchestrator.add_member(
    self.after_sales, AgentRole.EXECUTOR, ["after_sales"]
)

def handle_customer(self, message):
    ticket = self.message_bus.create_ticket(message)
    category = self.reception.classify(message)

    if category == "support":
        result = self.orchestrator.assign_task(
            {"id": ticket.id, "task": message, "capabilities": ["support"]},
            self.support_member
        )
    else:
        result = self.orchestrator.assign_task(
            {"id": ticket.id, "task": message, "capabilities": ["after_sales"]},
            self.after_sales_member
        )
    return result

```

2 Agent实现

```

class ReceptionAgent:
    def __init__(self, message_bus):
        self.message_bus = message_bus
        self.llm = OpenAI(model="gpt-4")

```

```

def classify(self, message):
    prompt = f"""将客户问题分类:

{message}

类别: support (技术支持) / after_sales (售后) / billing (账单) """
    response = self.llm.generate(prompt)
    return response.strip().lower()

class SupportAgent:
    def __init__(self, message_bus):
        self.message_bus = message_bus
        self.llm = OpenAI(model="gpt-4")
        self.kb = KnowledgeBase()

    def handle(self, ticket):
        solution = self.kb.search(ticket.message)
        if not solution:
            solution = self.llm.generate(
                f"作为技术支持回答: {ticket.message}"
            )
        self.message_bus.send_response(ticket.id, solution)
        return solution

class AfterSalesAgent:
    def __init__(self, message_bus):
        self.message_bus = message_bus
        self.llm = OpenAI(model="gpt-4")

    def handle(self, ticket):
        response = self.llm.generate(
            f"作为售后客服回答: {ticket.message}"
        )
        self.message_bus.send_response(ticket.id, response)
        return response

```

 **多Agent客服优势：**专业分工、7x24不间断、并发处理。三个Agent各司其职，客服效率翻倍。

有问题欢迎留言。

 相关系列阅读

- Agent开发系列 25-26：可观测性 + 生产部署

下期预告：Agent开发系列 29-30 — 框架对比 + 终极总结

Agent开发系列 · 用代码理解AI

公众号：大前端工程师
公众号：大前端工程师
公众号：大前端工程师

框架对比

市面上有很多Agent开发框架：AutoGPT、LangChain、CrewAI.....它们核心理念是什么？代码有什么区别？这一篇来做个深度对比。

💡 **核心观点：**框架是工具，理解和原理才是根本。看完这篇，你就知道该选什么框架了。

1 AutoGPT

AutoGPT是最早的Agent框架之一，核心理念是"自主Agent"——给定目标，Agent自己思考、分解、执行。

```
PYTHON

# AutoGPT核心逻辑 (伪代码)
class AutoGPT:
    def think(self, goal):
        # 1. 分析目标, 生成任务列表
        tasks = self.decompose(goal)

        # 2. 逐个执行任务
        for task in tasks:
            result = self.execute(task)
            self.review(result) # 自我审查

        # 3. 如果目标没达成, 重新规划
        if not self.goal_achieved():
            self.think(goal) # 递归
```

2 LangChain

LangChain是目前最流行的框架，核心理念是"链式组合"——把LLM、工具、记忆串联成链。

```
PYTHON

# LangChain核心逻辑
from langchain import LLMChain, Agent
from langchain.agents import initialize_agent, Tool

# 1. 定义工具
tools = [
    Tool(
        name="Search",
        func=search_func,
        description="用于搜索信息"
    ),
    Tool(
        name="Calculator",
        func=calc_func,
        description="用于计算"
    )
]

# 2. 初始化Agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="zero-shot-react-description"
)

# 3. 执行
result = agent.run("帮我查一下特斯拉股价并计算市盈率")
```

3 CrewAI

CrewAI专注于多Agent协作，核心理念是"团队"——多个Agent扮演不同角色，协作完成任务。



PYTHON

```
# CrewAI核心逻辑
from crewai import Agent, Task, Crew

# 1. 定义Agent角色
researcher = Agent(
    role="研究员",
    goal="提供准确的研究数据",
    backstory="你是资深研究员"
)

writer = Agent(
    role="作家",
    goal="写出优质文章",
    backstory="你是资深编辑"
)

# 2. 定义任务
task1 = Task("研究AI最新进展", agent=researcher)
task2 = Task("撰写文章", agent=writer)

# 3. 组建团队并执行
crew = Crew([researcher, writer], tasks=[task1, task2])
result = crew.kickoff()
```

4 框架对比总结

框架	核心理念	优势	劣势
AutoGPT	自主Agent	全自动化、目标驱动	token消耗大、难以控制
LangChain	链式组合	生态完善、灵活性高	学习曲线陡、版本迭代快
CrewAI	多Agent协作	角色清晰、易于理解	功能相对单一

 **选框架建议：**学习原理用AutoGPT、生产项目用LangChain、多Agent协作用CrewAI。当然，最理想的是理解原理后自己造轮子。

终极总结

30篇文章，终于写到最后了。这一篇，我们来做个完整的总结，回顾这个系列的旅程。

 **核心观点：**从会用工具，到会造工具，这是这个系列最想传递的东西。希望你学完之后，不只是个Agent用户，而是一个Agent开发者。

1 三大系列回顾

系列	主题	核心能力
AI编程系列	用AI工具编程	Copilot、Cursor、Claude
AI Coding系列	用AI Coding	项目级开发能力
Agent开发系列	自己造Agent	系统设计能力

2 Agent能力矩阵

学完这个系列，你应该掌握了以下能力：

- **核心循环：**感知、思考、行动、反思
- **记忆系统：**工作记忆、对话记忆、知识记忆、技能记忆
- **工具系统：**Tool接口、Function Calling、安全机制
- **规划系统：**任务分解、执行引擎、动态调整
- **多Agent：**协作模式、消息总线、团队编排
- **生产部署：**可观测性、Docker、K8s

3 下一步

学完这个系列，你可以：

- 从零构建自己的Agent
- 深入研究某个模块（比如RAG、多Agent协作）
- 参与开源Agent项目
- 把Agent能力应用到实际工作中

最后送大家一句话：**不要只做工具的用户，要做工具的创造者。**

💡 **写在最后：**三个系列到此结束。从会用、到会用工具、到会造工具——这个旅程，我们一起走完了。希望这个系列能帮你打开新世界的大门。

有问题欢迎留言。

📎 相关系列阅读

- AI Coding系列：[AI Coding通关指南](#)
- Agent开发系列 01-02：[从用户到创造者](#)

Agent开发系列 · 用代码理解AI · 完结

Agent开发系列 · 终极总结

30篇文章完整回顾——从用户到创造者，从工具到底层原理

四个系列的完整旅程

OpenClaw → AI Coding → Agent开发 → 大模型原理

从平台使用 → 工具掌握 → 应用创造 → 底层理解

1 为什么写这个系列

2024年初，AI应用开发的门槛还很高。大多数人还在问"AI能做什么"，而我已经决定回答另一个问题：["AI应用是怎么工作的"](#)。

这30篇文章，不是教你调API的教程，而是[从零理解AI应用内核](#)的完整路径。

核心目标：让每个读完这个系列的人，不仅会用Agent，还能理解Agent为什么这样工作——以及，当它不工作时，该怎么改。

2 四个系列的脉络梳理

让我们把四个系列串起来看：

系列	核心问题	学会了什么	解决了什么
OpenClaw	怎么用平台？	Bot设计、插件开发、工作流	从0到1跑起AI应用

AI Coding	怎么用AI写代码？	项目开发、架构设计、调试技巧	让AI成为开发助手
Agent开发	Agent是怎么工作的？	执行引擎、记忆、工具、多Agent	从用户变成开发者
大模型原理	LLM为什么会这样？	Transformer、Attention、GPT进化	从表象深入本质

四个系列形成了一个完美的学习闭环：

- 1. OpenClaw让你看见AI应用长什么样
- 2. AI Coding让你使用AI工具提升效率
- 3. Agent开发让你理解AI应用的内部构造
- 4. 大模型原理让你洞察构造背后的底层逻辑

3 Agent开发系列核心能力图谱

核心循环

感知 → 思考 → 行动 → 反思

Agent的四大基本能力

记忆系统

工作记忆、对话记忆

知识记忆、技能记忆

工具系统

Tool接口、Function Calling

安全机制、错误处理

规划系统

任务分解、执行引擎

动态调整、回退策略

多Agent协作

消息总线、团队编排

角色定义、结果聚合

生产部署

可观测性、Docker容器化

K8s编排、灰度发布

4

Agent开发的六个关键认知

这30篇文章里，我最想传递的，不是某个具体实现，而是六个关键认知：

认知一：Agent不是魔法

Agent的本质是一个**循环执行**的系统：感知输入 → 思考决策 → 调用工具 → 观察结果 → 继续循环。它之所以看起来"智能"，是因为大模型提供了强大的理解能力。

认知二：Prompt是接口，不是指令

很多人把Prompt当成"命令"来写，其实Prompt是**接口设计**。好的Prompt要定义清楚：Agent是谁、要做什么、怎么做、边界在哪。

认知三：记忆是有代价的

上下文窗口是有限的，Token是花钱的。记忆系统设计的目标是：**在有限资源下，让Agent记住最该记住的。**

认知四：工具调用是双刃剑

工具让Agent能力边界极大扩展，但也带来了**错误放大**的风险。Agent执行100步，只要一步出错，结果就可能偏离。所以工具设计要简洁、安全、有边界。

认知五：多Agent不是银弹

多Agent看起来很美——分工明确、各司其职。但协作成本很高：消息协议、状态同步、结果冲突.....很多场景下，单Agent反而更简单有效。

认知六：工程能力决定落地

Prompt调得再好，生产环境跑不起来也是白搭。可观测性、容错机制、资源控制.....这些"不性感"的东西，才是Agent能否上线的关键。

5 从Agent开发到大模型原理

学完Agent开发，你可能会问一些问题：

- 为什么上下文窗口那么大，Agent还是容易"忘"？
- 为什么RAG听起来简单，用起来却效果不稳定？
- 为什么同样的模型，别人的Agent就是比你的聪明？
- 为什么有些问题模型能答，有些却"一本正经地胡说八道"？

这些问题，不在Agent层解决，而在更底层——在大模型层。

为什么Agent会"智障"？

不是Agent设计有问题，而是大模型的能力边界在起作用。理解了大模型原理，你才能真正设计出"聪明"的Agent。

大模型原理系列（即将开启）将解答：

Agent开发中的困惑	大模型原理中的答案
上下文窗口怎么工作的？	Transformer的Attention机制

Embedding是怎么算的？	词向量与语义空间
为什么模型有"个性"差异？	GPT系列进化史与RLHF对齐
RAG怎么检索更准？	向量数据库与相似度计算
Prompt怎么写更有效？	Attention与Token权重
模型安全怎么保障？	对齐技术与安全边界

下个系列：大模型原理系列

- Transformer架构——从RNN到Attention
- Attention机制——Query/Key/Value详解
- GPT进化史——从GPT-1到GPT-5
- RLHF与对齐——让模型更"听话"
- Embedding与向量检索——语义搜索的原理
- Tokenizer与Token优化——精准控制成本

6 给读者的话

30篇文章写下来，最大的感受是：[AI应用开发](#)，是一个"knowing by building"的过程。

看一百遍原理，不如亲手写一个Agent跑起来。但跑起来之后，你会发现新的问题，然后你回来找原理——这个循环，才是真正理解AI应用的路径。

"不要只做工具的用户，要做工具的创造者。"

——这个系列最想传递的一句话

四个系列到此结束。但对你的AI开发之旅来说，这只是[真正的起点](#)。

学完这四个系列，你不再是"调API的程序员"，而是能从底层理解AI应用、从顶层设计系统的[AI应用开发专家](#)。

终极目标：当你面试时被问到"GPT是怎么生成文字的"，你能从Transformer讲到Attention，从Attention讲到自注意力——不是背答案，而是真正理解。

有问题欢迎留言，我们下个系列见。

四大系列阅读路径

- OpenClaw系列：平台使用 → Bot设计 → 插件开发
- AI Coding系列：工具使用 → 项目开发 → 工程能力
- Agent开发系列：执行引擎 → 记忆系统 → 工具调用 → 多Agent协作 → 生产部署
- 大模型原理系列（预告）：Transformer → Attention → GPT进化 → RLHF → Embedding

Agent开发系列 · 用代码理解AI · 完结

下一站：大模型原理系列